大模型算法与 Agent 开发面试指南

Nurdun Mijit

2026 年 6 月 8 日

目录

1 大模型基础与 Transformer 架构

1. Transformer 中因果注意力 (Causal Attention) 和双向注意力 (Bidirectional Attention) 的区别？

核心区别：

- **因果注意力（单向）：** 用于 Decoder-only 架构（如 GPT 系列）。在计算注意力时，当前 token 只能看到它之前的 token。通过上三角掩码（Mask）实现，确保模型在预测第 t 个 token 时无法“偷看”到 $t+1$ 及以后的信息。这是自回归生成的基础。
- **双向注意力：** 用于 Encoder 架构（如 BERT）。每个 token 可以关注序列中的所有其他 token。这种方式能捕捉更丰富的上下文语义，适合理解类任务（如分类、NER），但不适合直接生成文本。

2. RoPE (Rotary Positional Embedding) 为什么能实现位置编码？

原理深度解析： RoPE 的核心思想是通过绝对位置编码的方式实现相对位置编码的效果。

1. **旋转矩阵：** 它将向量视为复数，根据 token 的位置 m 进行旋转。对于维度 $2i$ 和 $2i+1$，旋转角度为 $\theta_i = 10000^{-2i/d}$。
2. **相对位置保持：** 两个向量 q_m 和 k_n 的内积只取决于它们的相对距离 $m-n$。数学表达为：
$$\langle f(q,m), f(k,n) \rangle = g(q,k,m-n)$$
3. **外推性优势：** 相比 Sinusoidal 编码，RoPE 在长序列外推（Extrapolation）时表现更好，因为其衰减特性更符合自然语言的相关性随距离衰减的规律。

3. 为什么现在 Decoder-only 成为大模型的主流架构？

- **通用性强：** 既能做生成（Native 能力），也能通过 Prompt 做理解和分类。
- **推理效率：** 在自回归生成场景下，Decoder-only 可以利用 KV Cache 极大加速推理，而 Encoder-Decoder 在生成时仍需处理完整的 Cross-Attention。
- **规模化效应：** 随着数据量和参数量的增加，Decoder-only 的性能上限更高，且训练流程更统一（纯 Next Token Prediction）。

2 参数高效微调 (PEFT) 与对齐技术

1. 在 PEFT 中，为什么优先微调 Attention 层的 Q/K/V/O，而不是 FFN 层？

深度原理解析：

- **知识存储 vs. 任务适配：** 研究表明，FFN 层主要存储了模型的“事实性知识”（Knowledge Storage），而 Attention 层负责“上下文推理”和“任务逻辑”。
- **梯度敏感度：** 在微调初期，Attention 层的参数对特定任务的损失函数下降贡献更大。微调 Q/K/V 可以改变模型对输入信息的关注分布，从而快速适配新领域。
- **实验结论：** 许多论文（如 LoRA 原始论文）对比发现，仅微调 Attention 模块能达到全量微调 90% 以上的效果，且参数量更少。

2. LoRA 与 QLoRA 的核心区别与优势

- **LoRA (Low-Rank Adaptation):** 冻结预训练权重，在旁路注入低秩矩阵 A 和 B。优点是训练快、显存占用中等；缺点是仍需加载全精度基座模型。
- **QLoRA:** 在 LoRA 基础上引入了 **4-bit NormalFloat (NF4)** 量化。
  1. 将基座模型量化为 4-bit。
  2. 使用双量化（Double Quantization）减少量化常数的开销。
  3. 使用分页优化器（Paged Optimizers）防止梯度检查点时的显存峰值。
- **优势：** QLoRA 使得在单张消费级显卡（如 RTX 3090/4090）上微调 65B 甚至更大的模型成为可能。

3. RLHF 中的 PPO 与 DPO 对比

特性	PPO (Proximal Policy Optimization)	DPO (Direct Preference Optimization)
训练方式	需要训练 Reward Model (RM)，再进行 RL 优化	直接在偏好数据上优化策略模型
稳定性	较差，超参数敏感，容易崩溃	极好，等价于交叉熵损失
资源消耗	极高（需同时加载 Actor, Critic, RM, Reference）	低（只需加载 Policy 和 Reference）
核心公式	基于奖励信号的优势函数估计	基于 Bradley-Terry 模型的似然比

3 RAG 系统与 GraphRAG 深度解析

1. RAG 的标准数据流程

1. **数据处理 (ETL):** 提取文档内容，进行清洗、分块 (Chunking)。
2. **向量化 (Embedding):** 使用 Embedding 模型将 Chunk 转化为高维向量。
3. **索引构建 (Indexing):** 存入向量数据库（如 Milvus, FAISS, ES）。
4. **检索 (Retrieval):** 用户 Query 向量化，召回 Top-K 相关片段。
5. **增强生成 (Generation):** 将 Query + Context 组装 Prompt，输入 LLM 生成答案。

2. RAG 与 GraphRAG 在多跳推理上的根本区别

传统 RAG 的局限： 依赖向量相似度。如果答案分散在不同文档的不同段落，且没有明显的关键词重叠，向量检索很难一次性召回所有必要信息（即“多跳”失败）。

GraphRAG 的优势：

- **结构化关联：** 通过抽取实体（Entity）和关系（Relation）构建知识图谱。
- **图遍历：** 可以通过图算法（如 PageRank, Shortest Path）发现实体间的隐性联系。
- **社区摘要：** GraphRAG 会对图中的社区（Community）生成摘要，使得模型能从全局视角回答宏观问题，而不仅仅是局部片段匹配。

3. 如何处理文档中的异构数据（图片/表格）？

处理流程：

1. **解析：** 使用 OCR 或专门的解析工具（如 MinerU, Nougat）提取图片和表格。
2. **跨模态理解：** 调用多模态大模型（如 Qwen-VL, GPT-4V）对图片进行详细描述（Captioning）。
3. **对齐与存储：** 将图片描述作为文本存入向量库，并保留图片原始路径。
4. **召回增强：** 检索时，如果用户的 Query 涉及视觉信息，不仅召回文本，还会召回对应的图片描述。

关于信息丢失： 单纯返回文字确实会丢失细节。工业界通常采用“图文混排”的返回方式，或者在 RAG 系统中增加一个“看图”的工具调用分支。

4 Agent 架构与工程实践

1. Agent 的核心架构 (ReAct / Plan-and-Solve)

- **感知 (Perception):** 接收用户指令，理解意图。
- **规划 (Planning):** 将复杂任务拆解为子任务（Sub-tasks）。常用方法：CoT (Chain of Thought), ToT (Tree of Thoughts)。
- **工具使用 (Tool Use):** 决定调用哪个 API（搜索、计算器、代码解释器）。
- **行动 (Action):** 执行工具调用。
- **反思 (Reflection):** 观察执行结果，判断是否完成任务，若失败则修正计划。

2. 如何限制工具调用次数与防止死循环？

- **硬约束：** 在代码层面设置最大迭代次数（Max Iterations），超过即终止。
- **软约束：** 在 System Prompt 中明确告知模型“请在 3 步内解决问题”。
- **状态检测：** 记录历史动作，如果发现连续两次调用相同的工具且参数相同，强制打断。

3. 什么是 SDD (Spec-Driven Development)?

SDD 是一种面向智能体的开发范式。传统的开发是写代码实现功能，而 SDD 是先定义清晰的 **规格说明书 (Spec)**，包括输入输出规范、工具定义、成功标准等。Agent 根据 Spec 自动寻找满足条件的实现路径。这解决了 Agent 行为不可控的问题。

5 推理加速与底层优化

1. KV Cache 的原理与优化

原理： 在自回归生成过程中，之前 token 的 Key 和 Value 向量是不会改变的。KV Cache 将这些中间结果缓存起来，避免每次生成新 token 时都要重新计算整个序列的 Attention。

优化技术：

- **PagedAttention (vLLM):** 借鉴操作系统的虚拟内存管理，解决 KV Cache 显存碎片化问题。
- **Quantized KV Cache:** 将 KV Cache 量化为 INT8 或 FP4，减少显存占用。

2. FlashAttention 的核心思想

痛点： 传统 Attention 的复杂度是 $O(N^2)$，且需要在 HBM（高带宽内存）和 SRAM（片上缓存）之间频繁读写巨大的中间矩阵 S 和 P。

FlashAttention 方案：

- **Tiling (分块):** 将矩阵切分成小块，使其能放入 SRAM。
- **Recomputation (重计算):** 在前向传播时不存储巨大的 Attention 矩阵，而是在反向传播时重新计算。
- **IO-Aware:** 极大地减少了 HBM 访问次数，实现了端到端的加速。

6 算法手撕与产品思维

1. 手撕代码：最长连续重复子序列 (LeetCode 674)

思路： 维护一个计数器 `count` 和一个最大值 `max_count`。遍历数组，如果当前元素等于前一个元素，`count++`，否则重置 `count=1`。

```python
def findLengthOfLCIS(nums: List[int]) -> int:
    if not nums: return 0
    max_count = 1
    count = 1
    for i in range(1, len(nums)):
        if nums[i] > nums[i-1]:
            count += 1
            max_count = max(max_count, count)
        else:
            count = 1
    return max_count
```

2. 产品思维：AI 功能的“不确定性”风险管理

- **上线前：** 建立“红队测试”（Red Teaming），专门攻击模型诱导其产生有害内容。构建 Golden Dataset 进行自动化评测。
- **线上监控：** 实时监控“拒绝率”、“用户举报率”以及“异常 Token 输出”。
- **应急响应：** 设计“一键降级”开关，一旦检测到大规模幻觉或安全漏洞，立即切换到规则-based 的传统客服或人工模式。